Simulation of Barrier Options Using Variance Reduction Techniques in Merton's Jump-Diffusion Framework

Chase Wiederstein

April 2025

Abstract

This paper investigates computational methods for pricing down-and-out barrier options under Merton's jump-diffusion model. Several Monte Carlo algorithms are explored, each incorporating different variance reduction techniques, including importance sampling, stratified sampling, and conditional expectation. Numerical experiments compare these methods in terms of accuracy, standard deviation, and computational efficiency across a range of jump intensities.

1 Introduction

It is well known that the classical Black-Scholes model has key flaws, particularly its assumption of geometric Brownian motion with constant volatility. Real market data show that log returns have heavier tails and higher peaks (i.e., excess kurtosis) than that predicted by a normal distribution. Black-Scholes also assumes continuous price paths, but real markets often exhibit sudden jumps and discontinuities. In 1976, Merton introduced the jump-diffusion model in his paper, [4], where the stock price follows a geometric Brownian motion with jumps following a Poisson process.

2 Merton's Jump Diffusion Model

Merton was one of the first to extend the renowned Black-Scholes model by introducing a jump-diffusion process. His model is defined by the following stochastic differential equation:

$$\frac{dS(t)}{S(t^-)} = \mu\, dt + \sigma\, dW(t) + dJ(t) \tag{1}$$

where $S(t^-) = \lim_{u \to t^-} S(u)$ is the left-limit of the stock price.

This equation consists of two main components. The first term, $\mu\, dt + \sigma\, dW(t)$, models the continuous evolution of the stock price due to normal market fluctuations. The second term, $dJ(t)$, captures discontinuous jumps in the price caused by rare or impactful events, such as breaking news, economic shocks, or policy changes.

In Equation (1), μ and σ represent the drift and volatility of the stock, respectively, as in the standard geometric Brownian motion model. $W(t)$ is a standard Brownian motion and the process $J(t)$ is a pure jump process, independent of $W(t)$, defined as:

$$J(t) = \sum_{i=1}^{N(t)} (Y_i - 1)$$

where $N(t)$ is a Poisson process with intensity λ, and Y_i are i.i.d. positive random variables representing jump multipliers.

Suppose a jump occurs at time τ_j, and consider the increment $dJ(t) = J(t+dt) - J(t)$. If no jump occurs in the interval $[t, t+dt]$, then $dJ(t) = 0$. If a jump occurs at $\tau_j = t + dt$, then:

$$dJ(t) = J(\tau_j) - J(t) = \sum_{i=1}^{N(\tau_j)} (Y_i - 1) - \sum_{i=1}^{N(t)} (Y_i - 1) = Y_j - 1$$

Assuming dt is sufficiently small, the probability of multiple jumps occurring in the interval $[t, t + dt]$ is negligible. Hence, the Poisson increment $dN(t)$ satisfies:

$$dN(t) = \begin{cases} 1 & \text{if } \tau_j \in [t, t + dt] \\ 0 & \text{otherwise} \end{cases}$$

Following the approach of Joshi and Leung in [3], we modify the drift term to ensure that the discounted stock price $e^{-rt}S(t)$ is a martingale under the risk-neutral measure [1]. This requires setting $\mu = r+\lambda(1-m)$, where $m = \mathbb{E}[Y]$ is the expected jump multiplier and r is the risk-free interest rate. The risk-neutral version of Merton's model becomes:

$$\frac{dS(t)}{S(t^-)} = (r + \lambda(1 - m))\ dt + \sigma\, dW(t) + (Y - 1)\, dN(t) \tag{2}$$

We assume that the jump multipliers Y follow a lognormal distribution given by:

$$Y = m\, e^{-\frac{1}{2}\sigma_{\text{jump}}^2 + \sigma_{\text{jump}} Z}$$

where Z is a standard normal random variable and σ_{jump} is the volatility of the jump size.

To understand how jumps affect the stock price, consider what happens at the jump time τ_j. By definition:

$$\frac{S(\tau_j) - S(\tau_j^-)}{S(\tau_j^-)} = Y_j - 1$$

which implies:

$$S(\tau_j) = S(\tau_j^-)Y_j$$

Thus, each jump acts as a multiplicative scaling of the stock price by a random factor Y_j. To avoid confusion, let $\log S(\tau_i) = X(\tau_i)$, and I will introduce the notation $X(\tau^+)$ to represent post-jump prices, and $X(\tau_i^-)$ will represent pre-jump prices. We can now express the stock value in log space as

$$X(\tau_i^+) = X(\tau_i^-) + \log m - \frac{1}{2}\sigma_{jump}^2 + \sigma_{jump} Z$$

where Z is a standard normal random variable. Setting $X(0) = \log S(0)$, then between jumps we can simulate the process which follows a geometric Brownian motion with

$$X(\tau_i^-) = X(\tau_{i-1}^+) + \nu(\tau_i - \tau_{i-1}) + \sigma(W(\tau_i) - W(\tau_{i-1}))$$

where $\nu = \mu - \frac{1}{2}\sigma^2$. Furthermore, taking $t_0 = 0$, we can simulate the jump times using the fact that $\Delta\tau_i = \tau_i - \tau_{i-1}$ is an exponential random variable.

3 Simulation Algorithms

In the following section, I will present several different approaches in which we can price a down-and-out option. Simply put, a down-and-out option is killed if the stock ever drops below the barrier H and has payoff if the stock is greater than strike K by expiry, T.

Crude Monte Carlo

A common approach to simulating stock prices is the "short-step method," where the time interval $[0, T]$ is partitioned into evenly spaced steps of size dt. Crude Monte Carlo refers to simply estimating the average payoff without applying any optimization techniques like importance sampling, control variates, etc. The algorithm presented in this section serves as a baseline for comparing the performance of more advanced Monte Carlo methods discussed later.

I first partition $[0, T]$ into a uniform mesh with time steps of size dt. Then, between successive points t_i and t_{i+1}, I generate $N(t)$, a Poisson random variable with rate λdt representing the number of jumps that have occurred within the interval. If a jump occurs, I update the log stock price according to $X(t_{i+1}) = X(t_i) + \nu dt + \sigma\sqrt{dt}z^{(1)} + (\log m - \frac{1}{2}\sigma_{jump}^2 + \sigma_{jump}z^{(2)})$ where $z^{(1)}$ and $z^{(2)}$ are standard normal random variables. If no jump occurs in the interval, I omit the jump term and only simulate the diffusion component. If any price drops below H, the path is killed. It is important to note that this method introduces an inherent discretization bias because the barrier is only monitored at discrete time points. This creates the possibility of missing barrier breaches between monitoring dates, leading to a biased estimate.[1]

Metwally and Atiya's Implementation

Metwally and Atiya take a different approach than seen from crude Monte Carlo. They first simulate jump arrival times which serve as the only points computations need to be done. This approach was found to be much faster than the short-step method.

The simulation framework of Metwally and Atiya (see [5] for details) is based on the Brownian bridge technique. The method begins by simulating all jump times $0 < \tau_1 < \tau_2 < \cdots < \tau_n < T$ from an exponential distribution. Between jumps, the stock price evolves according to a diffusion process, with increments drawn from a normal distribution. At each jump time, the stock log-price $X(\tau_i^+)$ for some $i \in [1, n]$ is updated by adding a lognormally distributed jump size to the pre-jump log-price $X(\tau_i^-)$.

Lemma 1. *Let $\{W(t)\}$ be Brownian motion with drift μ and volatility σ. Given $X(\tau_{i-1}^+) > \ln H$ and $X(\tau_i^-) > \ln H$*

$$\begin{aligned} P_i &= P\left(\inf_{\tau_{i-1} \le s \le \tau_i} W(s) > \ln H \mid W(\tau_{i-1}^+) = X(\tau_{i-1}^+), W(\tau_i^-) = X(\tau_i^-)\right) \\ &= 1 - \exp\left(\frac{-2\left[\log H - X(\tau_{i-1}^+)\right]\left[\log H - X(\tau_i^-)\right]}{(\tau_i - \tau_{i-1})\sigma^2}\right) \end{aligned}$$

Between each pair $X(\tau_i^+)$ and $X(\tau_{i+1}^-)$, we check whether either value has breached the barrier $\log H$. If either has, we kill the option and return a payoff of 0. Otherwise, we apply Lemma 1 to compute the probability that the Brownian path has crossed the barrier in between and reject the path accordingly. Note that the original paper and algorithm use a rebate of 1, but here we assume a rebate of 0. For a more detailed description of the entire algorithm, see the appendix.

Joshi and Leung's Implementation

In [3], Joshi and Leung extended the work of Metwally and Atiya by introducing importance sampling, a variance reduction technique commonly used in Monte Carlo simulations. Their approach reduced the frequency of zero payoffs by changing the underlying transition densities used in the simulation of asset paths

To formalize this, consider the stock path $\omega = S(t_0), S(t_1), ..., S(t_k)$ with known transition densities f_i. The probability of ω occurring is $f_1(S(t_0), S(t_1)) \times f_2(S(t_1), S(t_2) \times ... \times f_k(S(t_{k-1}), S(t_k))$. In many problems in finance, we often simulate a price path using the recursion $S(t_{i+1}) = G(S(t_i), X_{i+1})$ where the random vectors $X_1, X_2, ...$ are i.i.d and from a common distribution. Under the Black-Scholes model, these random vectors will be normal, enabling a more straightforward likelihood ratio as $\Pi_{i=1}^k \frac{f(X_i)}{g(X_i)}$ where g is the new common density after the change of measure[6]. We can now simulate the expected payoff, $h(\omega)$ using

$$\begin{aligned}\mathbb{E}(h(\omega)) &= \sum_{\omega} h(\omega) f_1(S(t_0), S(t_1)) \times ... \times f_k(S(t_{k-1}), S(t_k)) \\ &= \sum_{\omega} \left(h(\omega) \Pi_{i=1}^{k} \frac{f_i(S(t_{i-1}), S(t_i))}{g_i(S(t_{i-1}), S(t_i))} \right) g_1(S(t_0), S(t_1)) \times ... \times g_k(S(t_{k-1}), S(t_k)) \\ &= \tilde{\mathbb{E}} \left(h(\omega) \Pi_{i=1}^{k} \frac{f_i(S(t_{i-1}), S(t_i))}{g_i(S(t_{i-1}), S(t_i))} \right)\end{aligned}$$

However, for Merton's Jump diffusion model, the paths depend on both a diffusion term and a jump term, adding more complexity to the transition probabilities. The paths for Joshi and Leung's implementation take the form $\omega = S(\tau_0^+), S(\tau_1^-), S(\tau_1^+), ..., S(\tau_k^-), S(T)$.

When simulating the diffusion term, the aim is to produce the next stock price in log space, $X(\tau_{i+1}^-) = X(\tau_i^+) + \nu \Delta\tau_i + \sigma\sqrt{\Delta\tau_i} z^{(1)}$, such that the down-and-out call option is not killed. This means we need the normal random variable $z^{(1)} > \frac{\log H - X(\tau_i^+) - \nu\Delta\tau_i}{\sqrt{\Delta\tau_i}}$. Similarly, when simulating jump term, $X(\tau_{i+1}^+) = \log m - \frac{1}{2}\sigma_{jump}^2 + \sigma_{jump} z^{(2)}$, we enforce the same restriction such that $z^{(2)} > \frac{\log H - \log m + \frac{1}{2}\sigma_{jump}^2}{\sigma_{jump}}$. We now sample $z^{(1)}$ and $z^{(2)}$ from a truncated normal distribution, $\phi(z)/\theta$ instead of the original normal distribution $\phi(z)$ where $\theta = \mathbb{P}(z > \text{threshold})$. Here, "threshold" just refers to the bounds we computed above — the values that would otherwise immediately kill the option. In this set up, each random draw of $z^{(1)}$ and $z^{(2)}$ introduces its own likelihood ratio θ, so the final likelihood correction will be a product of all θ factors across the path.

Recall that jump inter-arrival times are exponentially distributed with rate λ. For small λ, the probability of observing jumps is low, making standard simulation inefficient for studying jump effects. To address this, Joshi and Leung force a jump to occur by using the inverse transformation method, scaling by the probability $\mathbb{P}(\tau_1 < T) = 1 - e^{-\lambda T}$. After forcing the first jump, all subsequent inter-arrival times are sampled normally as exponential random variables. Because only the first jump is forced, the final payoff needs to be corrected by a likelihood ratio of $\mathbb{P}(\tau_1 < T) = 1 - e^{-\lambda T}$.

Another technique used is very similar to a method by Metwally and Atiya. Between each pair of $X(\tau_{i-1}^-)$ and $X(\tau_i^+)$, the probability of survival, $P_{\text{survive}} = 1 - P_{\text{breach}}$, can be computed using Lemma 1. Note that there is a typo stating Lemma 1 computes the probability of breaching, not survival, in the original paper [3]. For each path, we attach a weight of $1 - P_{\text{survive}}$ to each bridge and then multiply all the breach probabilities together to adjust the final payoff. This way paths with high probabilities of breaching contribute less to the payoff instead of not contributing to the average payoff.

Finally, Joshi and Leung decompose their final price estimate condition on whether a jump occurs or not. Let $C(S,t)$ denote the price of the barrier option

$$C(S(0),0) = \mathbb{P}(\tau_1 > T) C_0^{\text{No Jump}}(S(0),0) + \mathbb{P}(\tau_1 \leq T) C_0^{\text{Jump}}(S(0),0)$$

Where $C_0^{\text{No Jump}}$ denotes the price of the option conditioned on no jumps occurring, and C_0^{Jump} is conditional on jumps occurring before expiry. So far, the importance sampling and likelihood ratio techniques mentioned above are all used to price the $\mathbb{P}(\tau_1 \leq T) C_0^{\text{Jump}}$. As for when no jump occurs, we can use the following expression to analytically evaluate the price.

$$\begin{aligned}C_{\text{NoJump}}(S_0,0) = S_0 e^{(1-m)\lambda T} &\left(\Phi\left(\frac{\tilde{\nu}T + \log\left(\frac{S_0}{K}\right)}{\sigma\sqrt{T}} \right) - \left(\frac{H}{S_0}\right)^{2\tilde{\nu}\sigma^{-2}} \Phi\left(\frac{\log\left(\frac{H^2}{KS_0}\right) + \tilde{\nu}T}{\sigma\sqrt{T}} \right) \right) \\ - e^{-rT} K &\left(\Phi\left(\frac{\nu T + \log\left(\frac{S_0}{K}\right)}{\sigma\sqrt{T}} \right) - \left(\frac{H}{S_0}\right)^{2\nu\sigma^{-2}} \Phi\left(\frac{\log\left(\frac{H^2}{KS_0}\right) + \nu T}{\sigma\sqrt{T}} \right) \right)\end{aligned} \tag{3}$$

Note that Φ is the standard normal cumulative distribution function and $\tilde{\nu} = \nu + \sigma^2$. For all details of the implementation of Joshi and Leung, the complete algorithm is shown in the appendix.

Ross and Ghamami's Implementation

Ross and Ghamami in, [7], also furthered the work of Metwally and Atiya. Note that in the original paper, this algorithm was used to calculate an up-and-out call option and I have molded it to fit a down-and-out call option. They implement stratified sampling stratifying on the number of jumps, $N(T)$, then using conditional expectation. They compute the discounted stock option using the following:

$$\mathbb{E}(C(S(0),0) = \sum_{m=0}^{q} \mathbb{E}(C(S(0),0)|N(T)=m)P_m + \mathbb{E}(C(S(0),0)|N(T)>q)\bar{P}_q$$

where q is chosen to be small or such that $q \geq \lambda T + 3\sqrt{\lambda}T$, $P_m = \mathbb{P}(N(T)=m)$, and $\bar{P}_q = \mathbb{P}(N(T)>q)$

There are essentially three cases to evaluate the expected payoff. For the first case, if $N(T)=0$, we will simply use equation (3) since the expected payoff is analytically computable.

For the second case, we will need to estimate the expected payoff for $0 < m \leq q$. To do so, we first randomly generate m jumps using the lognormal distribution and compute $Y = \Pi_{i=1}^{n} Y_i$. Next, we compute the final payoff as $S(T) = S(0)Ye^W$ where $W = \nu T + \sigma\sqrt{T}z$ for some standard normal random variable. We force $S(T) > K$ by drawing the normal random variable $z > \frac{\log(K/(S_0 Y)) - \nu T}{\sigma\sqrt{T}}$ from a truncated distribution and multiplying the final payoff by the likelihood ratio similar to what was seen in the previous section. We will use the following lemma to construct the intermediate stock values in between 0 and T.

Lemma 2. *Let Y_i for $i = 1, ..., n$ be independent normal random variables with respective means μ_i and variance σ_i^2. Let $W = \sum_{i=1}^{n} Y_i$. If $\tilde{Y}_i$ has the distribution of Y_i given that $W = w$ and $Y_1 = \gamma_1, Y_2 = \gamma_2, ..., Y_{i-1} = \gamma_{i-1}$, then $\tilde{Y}_i$ is a normal random variable with mean and variance*

$$\tilde{\mu_i} = \mu_i + \frac{\sigma_i^2 \left(w - \sum_{j=1}^{i-1} \gamma_j - \sum_{j=i}^{n} \mu_j\right)}{\sum_{j=i}^{n} \sigma_j^2}$$

$$\tilde{\sigma}_i^2 = \sigma_i^2 \left(1 - \frac{\sigma_i^2}{\sum_{j=i}^{n} \sigma_j^2}\right)$$

Note that the original paper had an error and [2] derived the correct formula. For all pairs of $S(\tau_{i-1})^+$ and $S(\tau_i^-)$, the probability of the stock breaching the barrier is calculated with lemma 1. For all stock values, we monitor whether the barrier has been breached and kill the option if so. A likelihood ratio of the product of survival probabilities is then applied to the final payoff estimate.

As for case three, we use the inverse transformation method to generate a value X which is Poisson with rate λT, conditional on the event $X > q$. We then apply the steps followed in the second case to produce the estimate. Note that the complete algorithm has not been included in this paper.

Numerical Results

The following results are made to price a down-and-out call option. The parameters used are as follows: $S(0) = 100$, $K = 110$, $H = 95$, $\sigma = .25$, $\sigma_{jump} = .1$, $m = 1.005$, $T = 1$, and $r = .05$. For each algorithm, 30 trials of 1,000 simulations were carried out. Many more simulations would be necessary to increase reliability of results. The variance was averaged over each trial and the results in the tables below are for all trials. The time reported was the total time it took to compute all 30 trials for each method. For crude Monte Carlo, I partitioned the interval into 1,000 uniform steps. All simulations used Python 3.11.12 and the was a 12th Gen Intel(R) Core(TM) i7 - 12650H.

λ	Mean	Std. Dev.	Time Taken (s)
.1	4.275	4.474	15.9
.2	4.409	7.268	15.2
.5	4.930	8.478	18.4
1	5.202	10.159	23.4
2	5.638	13.170	26.6
4	5.619	15.591	30.2
8	5.677	17.577	30.0

Table 1: Ross and Ghamami

λ	Mean	Std. Dev.	Time Taken (s)
0.1	4.055	12.652	1.48
0.2	4.051	12.727	1.32
0.5	4.127	13.302	0.89
1	4.325	13.701	0.79
2	4.404	14.625	1.98
4	4.776	16.356	1.47
8	5.594	20.031	2.11

Table 2: Metwally and Atiya

λ	Mean	Std. Dev.	Time Taken (s)
0.1	4.067	.781	37.6
0.2	4.119	1.515	38.3
0.5	4.254	3.345	41.9
1	4.345	5.354	46.3
2	4.609	7.959	56.8
4	4.830	10.080	84.5
8	5.516	13.156	149.0

Table 3: Joshi and Leung

λ	Mean	Std. Dev.	Time Taken (s)
0.1	4.427	13.512	123.4
0.2	4.596	13.737	123.8
0.5	4.724	14.263	121.8
1	4.654	14.483	126.3
2	5.171	16.213	124.1
4	5.447	17.957	122.6
8	6.031	21.214	123.6

Table 4: Crude MC

The algorithms by Joshi and Leung, and Metwally and Atiya have been validated through comparisons with the results reported in [3]. Comparing the two, it is evident that Metwally and Atiya's method is significantly faster, but—as expected—it exhibits higher variance. The crude Monte Carlo was expected to have a higher mean payoff, which is confirmed in this experiment, although it required a considerably longer runtime. All algorithms outperform the crude algorithm in terms of computation time and variance. Regarding the method of Ross and Ghamami, the results show a lower standard deviation than that of Metwally and Atiya, as anticipated. Its computational time is also better than that of Joshi and Leung, indicating that it may offer an optimal trade-off between variance and efficiency. However, some observed bias may stem from my current implementation. I will continue to improve my results for the Ross and Ghamami algorithm.

References

[1] Paul Glasserman. *Monte Carlo methods in financial engineering*. Springer, 2004.

[2] Chace Gordon. Efficient monte carlo simulation of barrier option prices under the jump-diffusion framework. 2016.

[3] Mark Joshi and Terence Leung. Using monte carlo simulation and importance sampling to rapidly obtain jump-diffusion prices of continuous barrier options. *The Journal of Computational Finance*, 10:93–105, 06 2007.

[4] Robert C. Merton. On the Pricing of Corporate Debt: The Risk Structure of Interest Rates. *The Journal of Finance*, 29(2):449–470, 1974.

[5] Steve Metwally and Amir Atiya. Using brownian bridge for fast simulation of jump-diffusion processes and barrier options. *The Journal of Derivatives*, 10:43–54, 08 2002.

[6] Giray Ökten. Variance reduction techniques. Lecture notes, Department of Mathematics, Florida State University, 2025.

[7] Sheldon Ross and Samim Ghamami. Efficient monte carlo barrier option pricing when the underlying security price follows a jump-diffusion process. *The Journal of Derivatives*, page 100205044238096, 02 2010.

A Simulation Algorithms

Algorithm 1: Atiya-Metwally Down-and-Out Barrier Option Pricing

Input: $S_0, K, H, T, r, \sigma, \lambda, m, \sigma_{jump}, N$
Output: Option price estimate

Initialize total_payoff $\leftarrow 0$;
for $n = 1$ **to** N **do**
  Generate jump times $\tau_1, \ldots, \tau_K$ via inverse transformation method according to an exponential distribution with rate λ;
  for $i = 1$ **to** K **do**
    $\Delta\tau \leftarrow \tau_i - \tau_{i-1}$;
    $X(\tau_i^-) \leftarrow X(\tau_{i-1}^+) + \left(r - \lambda(m-1) - \frac{1}{2}\sigma^2\right)\Delta\tau + \sigma\sqrt{\Delta\tau}z^{(1)}$
    $X(\tau_i^+) \leftarrow X(\tau_i^-) + \log m - \frac{1}{2}\sigma_{jump}^2 + \sigma_{jump}z^{(2)}$
    for *Intervals* $i = 1$ **to** $K + 1$ **do**
      $P_i \leftarrow 1 - \exp\left(-\frac{2(\ln H - X(\tau_{i-1}^+))(\ln H - X(\tau_i^-))}{\sigma^2\Delta\tau}\right)$;
      $b \leftarrow \frac{\Delta\tau}{1-P_i}$;
      $u \leftarrow \mathcal{U}(\tau_{i-1}, \tau_{i-1} + b)$;
      if $\tau_{i-1} \le u \le \tau_i$ **then**
        Payoff $\leftarrow 0$
      if $X(\tau_i^+) \le \ln H$ **then**
        Payoff $\leftarrow 0$
      $i \leftarrow i + 1$
  if *No barrier hit* **then**
    Payoff $\leftarrow \max(e^{X(T)} - K, 0)e^{-rT}$;
  total_payoff $\leftarrow$ total_payoff + Payoff;
return *total_payoff*/N

Algorithm 2: Joshi-Leung Down-and-Out Barrier Option Pricing

Input: $S_0, K, H, T, r, \sigma, \lambda, m, \sigma_{\text{jump}}, N, \nu$

Output: Option price estimate

$P_{\text{Jump}} \leftarrow 1 - e^{-\lambda T}$;

Initialize total_payoff $\leftarrow 0$;

for $n = 1$ **to** N **do**

  Initialize likelihood ratio $L \leftarrow 1$;

  Sample first jump time τ_1 using truncated exponential with rate λ;

  Generate additional jump times $\tau_2, \ldots, \tau_K$ using standard exponential with rate λ;

  Set initial value $X(0) \leftarrow \log(S_0)$;

  for $i = 1$ **to** K **do**

    $\Delta\tau \leftarrow \tau_i - \tau_{i-1}$;

    $z^{(1)} \leftarrow \frac{\log H - X(\tau_{i-1}^+) - \nu\Delta\tau}{\sigma\sqrt{\Delta\tau}}$;

    $\theta \leftarrow 1 - \Phi(z^{(1)})$;

    Draw $u \sim U(0,1)$;

    $Z \leftarrow \Phi^{-1}(1 - \theta u)$;

    $X(\tau_i^-) \leftarrow X(\tau_{i-1}^+) + \nu\Delta\tau + \sigma\sqrt{\Delta\tau}Z$;

    $L \leftarrow L \times \theta$;

    Compute P_{breach} between τ_{i-1} and τ_i using Lemma 1;

    $L \leftarrow L \times (1 - P_{\text{breach}})$;

    $z^{(2)} \leftarrow \frac{\log H - \log m + \frac{1}{2}\sigma_{\text{jump}}^2}{\sigma_{\text{jump}}}$;

    $\theta \leftarrow 1 - \Phi(z^{(2)})$;

    Draw $u \sim U(0,1)$;

    $Z \leftarrow \Phi^{-1}(1 - \theta u)$;

    $X(\tau_i^+) \leftarrow X(\tau_i^-) + \log m - \frac{1}{2}\sigma_{\text{jump}}^2 + \sigma_{\text{jump}}Z$;

    $L \leftarrow L \times \theta$;

  $\Delta\tau \leftarrow T - \tau_K$;

  $z^{(1)} \leftarrow \frac{\log H - X(\tau_K^+) - \nu\Delta\tau}{\sigma\sqrt{\Delta\tau}}$;

  $\theta \leftarrow 1 - \Phi(z^{(1)})$;

  Draw $u \sim U(0,1)$;

  $Z \leftarrow \Phi^{-1}(1 - \theta u)$;

  $X(T) \leftarrow X(\tau_K^+) + \nu\Delta\tau + \sigma\sqrt{\Delta\tau}Z$;

  $L \leftarrow L \times \theta$;

  Compute P_{breach} between τ_K and T using Lemma 1;

  $L \leftarrow L \times (1 - P_{\text{breach}})$;

  Compute payoff with jump:

$$C_{\text{Jump}} \leftarrow e^{-rT} \max(e^{X(T)} - K, 0) \times L \times P_{\text{Jump}}$$

  Compute C_{NoJump} using the no-jump closed formula;

  total_payoff $\leftarrow$ total_payoff $+ C_{\text{Jump}} + (1 - P_{\text{Jump}}) \times C_{\text{NoJump}}$;

return *total_payoff/N*